

Prueba de desempeño Módulo Full Stack & Datos

Sistema de Pedidos y Analítica – EcoDelivery S.A.S.

La empresa ficticia **EcoDelivery S.A.S.** es una startup de domicilios ecológicos (bicicleta y moto eléctrica) que opera en cinco zonas de la ciudad: Norte, Sur, Centro, Occidente y Chapinero. Hasta ahora registran sus pedidos en una hoja de cálculo, lo que genera datos desactualizados, procesos manuales y falta de visibilidad para el equipo de operaciones.

La gerencia decidió que esto no puede seguir así. Por eso solicita el desarrollo de una solución compuesta por una **app móvil**, un **backend con API REST** (desarrollado con **Node.js + Express** o **FastAPI**, a tu elección), un **pipeline de datos en Airflow** y un **dashboard en Power BI**, que permitan centralizar y automatizar la operación de pedidos.

Tu misión en esta prueba es **construir esa solución**.

Puedes usar herramientas de IA como apoyo (autocompletado, dudas puntuales), pero debes poder **explicar y defender cualquier decisión técnica** en la revisión posterior. No es necesario que cada módulo esté “terminado” al 100%: se evalúa el avance, la calidad de lo entregado y la coherencia entre módulos.

1. Aplicación Móvil (Flutter)

La app está dirigida a clientes y repartidores de EcoDelivery para crear pedidos y actualizar su estado en tiempo real, consumiendo la API construida en el módulo 2.

Pantallas requeridas

- **Lista de pedidos:** consume GET /pedidos, muestra cliente, zona, estado (con color por estado) y monto. Debe permitir filtrar por estado o zona.
- **Detalle de pedido:** muestra toda la información y un botón para avanzar el estado (ej. de pendiente a en_camino, o de en_camino a entregado).
- **Crear pedido:** formulario que llama a POST /pedidos con validación básica de campos.

Requisitos

- Consumo real de la API (no datos fijos escritos en el código).
- Manejo de estados de carga y error (spinner, mensaje si la API falla).

Extra (opcional, no bloqueante)

- Pantalla de login simple.
- Pull-to-refresh en la lista de pedidos.

2. Backend / API REST

Construye una API REST que gestione los pedidos de EcoDelivery, con persistencia real y datos consistentes con los demás módulos.

Modelo de datos: Pedido

- **id_pedido** (número/string) — autogenerado.
- **cliente** (string) — requerido.
- **zona** (string) — uno de: Norte, Sur, Centro, Occidente, Chapinero.
- **fecha_creacion** (datetime) — autogenerado al crear.
- **fecha_entrega** (datetime) — nulo hasta que se marque como entregado.
- **estado** (string) — pendiente | en_camino | entregado | cancelado.
- **repartidor** (string) — nulo hasta asignarse.
- **metodo_pago** (string) — efectivo | tarjeta | app.
- **monto** (número) — requerido.

Endpoints requeridos

1. Crear pedido

POST /pedidos — crea un pedido con estado inicial pendiente.

2. Listar pedidos

GET /pedidos — debe soportar filtros por query params ?estado= y ?zona=.

3. Detalle de pedido

GET /pedidos/:id — devuelve el detalle de un pedido.

4. Actualizar estado

PATCH /pedidos/:id/estado — valida transiciones lógicas (ej. no pasar de cancelado a entregado). Si el nuevo estado es entregado, registra fecha_entrega automáticamente.

Requisitos

- Persistencia real (SQLite, PostgreSQL o similar) — no se acepta solo un arreglo en memoria.

- Validación de campos requeridos y códigos HTTP correctos (400, 404, 201, etc.).
- Datos de ejemplo disponibles para probar la API (puedes basarte en dataset_pedidos_semilla.csv).

Extra (opcional, no bloqueante)

- Autenticación simple (JWT o API key) en los endpoints de escritura.

3. Pipeline de Datos (Airflow)

EcoDelivery necesita un proceso que resuma la operación del día para el equipo de negocio.

Qué debe producir el DAG

Un DAG llamado etl_pedidos_diario, con al menos 3 tareas encadenadas (extracción, transformación, carga), que calcule:

- Tiempo promedio de entrega (en minutos) por zona, solo para pedidos entregado.
- Cantidad de pedidos por estado.
- Ingresos totales (monto) por zona.

Fuente y salida

- **Fuente:** tu propio backend (GET /pedidos) o dataset_pedidos_semilla.csv si no completaste el módulo 2.
- **Salida:** un archivo reporte_pedidos.csv (o una tabla nueva en base de datos) que será la fuente del dashboard del módulo 4.

Requisitos

- El DAG debe poder ejecutarse (manualmente desde la UI de Airflow o vía airflow dags test) y producir el archivo de salida.
- No es necesario dejarlo corriendo con scheduler real.

4. Dashboard (Power BI)

Construye un tablero para el equipo de operaciones de EcoDelivery, usando como fuente reporte_pedidos.csv (resultado del módulo 3) o dataset_pedidos_semilla.csv si no llegaste a Airflow.

Requisitos mínimos

Al menos 3 visualizaciones:



- Un gráfico de barras: pedidos o ingresos por zona.
- Un gráfico de líneas o área: pedidos por día.
- Una tarjeta (KPI) con el ingreso total o el ticket promedio.
- Al menos 1 segmentador (slicer) para filtrar por estado o por zona.
- Al menos 1 medida DAX creada por ti (no un simple conteo automático de Power BI).

Entregable

Archivo .pbix, o si no tienes licencia/instalación disponible, capturas de pantalla del tablero junto con el archivo de datos usado.

5. Entregables

Organiza todo en esta estructura (o equivalente) y entrégalo como repositorio Git o carpeta comprimida:

- backend/
- app_flutter/
- airflow/dags/etl_pedidos_diario.py
- powerbi/dashboard_ecodelivery.pbix (o capturas/)
- dataset_pedidos_semilla.csv (el que se te entregó, o el que generaste)
- README.md
- **IMPORTANTE NO OLVIDAR COLOCAR DENTRO DEL README:**
 - Cómo instalar dependencias y ejecutar cada módulo.
 - Qué decidiste dejar incompleto y por qué (si aplica).
 - Cualquier supuesto que hayas tomado sobre el caso de negocio.